

Chapter 5 – Building an Optimized Portfolio with R

Applying the mean-variance optimization approach (Markowitz's Modern Portfolio Theory) to your portfolio can be challenging. Often, people turn to Excel to set up the covariance table and run Solver to find the maximum Sharpe ratio (see the series [here](#) for instructions on using Excel). However, Excel is a poor choice as it takes a long time to download the data, setup the tables, and properly configure Solver. Fortunately, the R platform is a great tool for the job—and it is free.

In this article, we will show how you can get R configured quickly. Using the code below, you will be able to generate an optimal portfolio and map the efficient frontier in a matter of minutes—as compared with the lengthy (and error-prone) process using Excel.

This is by far a better, more powerful, and quicker approach. There is no excuse for not using the code below to test your own portfolio to identify potential opportunities for increasing your expected return while minimizing your portfolio's variance (risk).

You can access the code for all instructions below [here](#).

Introduction

A quick look at an efficient frontier reminds the reader that this is a parabolic curve and therefore finding the optimal portfolio is a task that involves finding the solution that minimizes a quadratic equation.

Since our objective is to provide an easy-to-use-guide to building an optimized portfolio, we will avoid delving too deep into the mathematics involved in solving quadratic formulas.

However, we will try to introduce basic concepts so that the reader can experiment with the tools—and has a base of knowledge on which to evaluate the results.

The steps involved in using R to build an optimized portfolio using the Markowitz Modern Portfolio Theory are outlined below.

1. Install R and the necessary packages
2. Download the return data
3. Solve for the Efficient Frontier
4. Map the efficient frontier

1. Install R and the necessary packages

If you have not already downloaded and installed R, now is the time to try it out. R is free and available for all major operating systems. Installing it takes less than 5 minutes in most cases and you can copy and paste the code from this article ([available here](#)) to get up and running quickly.

Begin by downloading the software for your operating system (Windows, MacOS, and Linux) from the [**Comprehensive R Archive Network site**](#).

One of the most helpful aspects of R is that contributors have written many add-on packages, which extend R's functionality and facilitates analysis by reducing the amount of code that needs to be written—especially for routine tasks. After you have downloaded and installed R, you will need to install several

additional packages. These packages are written by contributors and you'll find thousands of such add-ons to tackle just about every type of analytical problem you can imagine. For this article, you'll need to download the following:

- `stockPortfolio` – we use this to download stock data
- `quadprog` – we use this to solve for the optimal portfolio

After you have downloaded the packages, you will need to load them into your workspace (where R stores your environment) by issuing the following commands.

```
1 library(stockPortfolio)
2 library(quadprog)
```

That is it—you are ready to begin.

2. Download the return data

Although you can write your own R code to fetch and format stock returns, it is much easier to load and use the `stockPortfolio` package.

Begin by creating a list of the stocks you wish to include in your analysis. For this article, we have chosen to use the same six ETFs that represent a cross-section of equity and debt securities (same as we used in the **Excel based series**). Issue the following to assign the securities to the `stocks` object.

```
1 stocks <- c(
2   "SPY",
3   "EFA",
4   "IWM",
5   "VWO",
6   "LQD",
7   "HYG")
```

Loading the return data in R using the `stockPortfolio` package is extremely easy and can be done with one step. (This can involve a series of steps in Excel—and each step is an opportunity to introduce errors.)

```
1 returns <- getReturns(names(stocks), freq="week")
```

You can see a summary of the returns by issuing the following command. (We also suggest peeking at the structure of this object using the `str()` command.)

Summary of Returns

Watch the short video below for a demonstration on how to retrieve return data using R.



3. Solve for the Efficient Frontier

Unlike Excel, creating the efficient frontier is fast in R. After the initial code (demonstrated below) is set up, it takes only seconds to add securities, impose constraints, and compare data sets. Setting up the tables in Excel is time-consuming and cumbersome—and impractical for a portfolio with more than a few securities.

Additionally, as we'll demonstrate in another article, R allows for much greater control and sophistication in calculating covariance.

We are going to use a function (`solve.QP`) from the `quadprog` package we installed and loaded earlier. In order to effectively use this function (which can seem complex and a little overwhelming to those not steeped in quadratic theory), we need to understand the various arguments.

- **Dmat**: The covariance table (matrix). We will calculate this based on the data frame of returns.
- **dvec**: This is a vector of the average returns for each security. To find the minimum portfolio variance, we set these all to zero. To find points along the efficient frontier, we use a for loop to allow these returns to vary.
- **Amat**: This is the matrix of constraints. This can be a bit complicated—hang in there as we explain a little behind creating a matrix that imposes constraints. For those not steeped in algebraic matrix math, it may be easiest to learn by examining the examples. In the example, we use Amat to impose three distinct constraints: the portfolio weights must sum to 1, whether we allow any weights to be negative (implying short-selling and leverage), and whether there is a limit to any individual weight (to avoid high concentrations in just one security).
- **bvec**: Think of this as the legend to the Amat. This is a vector of values that is matched up against the Amat matrix to enforce our constraints. Again, if you are not familiar with matrix math, it may be easiest to learn by looking at the example and playing around with the code.
- **meq**: This simply tells the `solve.QP` function which columns in the Amat matrix to treat

as equality constraints. In our example, we only have one (the weights must sum to 1). The others are all inequality constraints (for example, weight must be > 0). We simply assign 1 to meq.

- **factorized:** We do not use this argument, you can ignore it.

We have created the function below that takes a data frame of security returns and several arguments and returns a data frame containing points along the efficient frontier. You can simply cut and paste this code into your R console. Or you can access the full code [here](#).

```

1  eff.frontier <- function (returns, short="no", max.allocation=NULL,
2  # return argument should be a m x n matrix with one column per secur
3  # short argument is whether short-selling is allowed; default is no
4  # max.allocation is the maximum % allowed for any one security (redu
5  # risk.premium.up is the upper limit of the risk premium modeled (se
6  # risk.increment is the increment (by) value used in the for loop
7
8  covariance <- cov(returns)
9  print(covariance)
10 n <- ncol(covariance)
11
12 # Create initial Amat and bvec assuming only equality constraint (sh
13 Amat <- matrix (1, nrow=n)
14 bvec <- 1
15 meq <- 1
16
17 # Then modify the Amat and bvec if short-selling is prohibited
18 if(short=="no"){
19   Amat <- cbind(1, diag(n))
20   bvec <- c(bvec, rep(0, n))
21 }
22
23 # And modify Amat and bvec if a max allocation (concentration) is sp
24 if(!is.null(max.allocation)){
25   if(max.allocation > 1 | max.allocation < 0){
26     stop("max.allocation must be greater than 0 and less than 1")
27   }
28   if(max.allocation * n < 1){
29     stop("Need to set max.allocation higher; not enough assets to add to
30   }
31   Amat <- cbind(Amat, -diag(n))
32   bvec <- c(bvec, rep(-max.allocation, n))
33 }
34
35 # Calculate the number of loops based on how high to vary the risk p
36 loops <- risk.premium.up / risk.increment + 1
37 loop <- 1
38
39 # Initialize a matrix to contain allocation and statistics
40 # This is not necessary, but speeds up processing and uses less memo
41 eff <- matrix(nrow=loops, ncol=n+3)
42 # Now I need to give the matrix column names
43 colnames(eff) <- c(colnames(returns), "Std.Dev", "Exp.Return", "shar
44
45 # Loop through the quadratic program solver
46 for (i in seq(from=0, to=risk.premium.up, by=risk.increment)){
47   dvec <- colMeans(returns) * i # This moves the solution up along the
48   sol <- solve.QP(covariance, dvec=dvec, Amat=Amat, bvec=bvec, meq=meq

```

```

49 eff[loop,"Std.Dev"] <- sqrt(sum(sol$solution * colSums((covariance *
50 eff[loop,"Exp.Return"] <- as.numeric(sol$solution %*% colMeans(retur
51 eff[loop,"sharpe"] <- eff[loop,"Exp.Return"] / eff[loop,"Std.Dev"]
52 eff[loop,1:n] <- sol$solution
53 loop <- loop+1
54 }
55
56 return(as.data.frame(eff))
57 }

```

The hard work is done! After you have this code loaded into R you can map the efficient frontier.

4. Map the Efficient Frontier

First, we run the function we just created against our returns and based on our portfolio constraints. For this example, we will assume that short-selling is allowed and that we impose a maximum allocation of 45% for any one security.

```

1 eff <- eff.frontier(returns=returns$R, short="yes", max.allocation=.4

```

Take a quick peek at what the function returned by using the `head()` function.

These are the portfolio weights.
Notice maximum % is 45%.

```
> head(eff)
```

	SPY	EFA	IWM	VWO	LQD	HYG	Std.Dev	Exp.Return	sharpe
1	0.4268875	-0.1175231	-0.1914503	-0.017914104	0.45	0.45	0.01425424	0.001442847	0.1012223
2	0.4300794	-0.1243912	-0.1909173	-0.014770882	0.45	0.45	0.01425447	0.001449533	0.1016897
3	0.4332714	-0.1312593	-0.1903844	-0.011627660	0.45	0.45	0.01425518	0.001456219	0.1021537
4	0.4364633	-0.1381274	-0.1898515	-0.008484438	0.45	0.45	0.01425635	0.001462905	0.1026143
5	0.4396553	-0.1449955	-0.1893185	-0.005341216	0.45	0.45	0.01425799	0.001469591	0.1030714
6	0.4428472	-0.1518637	-0.1887856	-0.002197994	0.45	0.45	0.01426010	0.001476277	0.1035250

Notice that the first six columns are the weights (or allocation percentages) of each security. The last three columns are statistics about that particular portfolio. We'll use standard deviation and expected return columns to create the efficient frontier.

Use the following code to create the efficient frontier.

```

1 eff.optimal.point <- eff[eff$sharpe==max(eff$sharpe),]
2
3 # Color Scheme
4 ealred <- "#7D110C"
5 ealtan <- "#CDC4B6"
6 eallighttan <- "#F7F6F0"
7 ealdark <- "#423C30"
8 ggplot(eff, aes(x=Std.Dev, y=Exp.Return)) + geom_point(alpha=.1, col
9 geom_point(data=eff.optimal.point, aes(x=Std.Dev, y=Exp.Return, lab
10 annotate(geom="text", x=eff.optimal.point$Std.Dev, y=eff.optimal.po
11 label=paste("Risk: ", round(eff.optimal.point$Std.Dev*100, digits=3

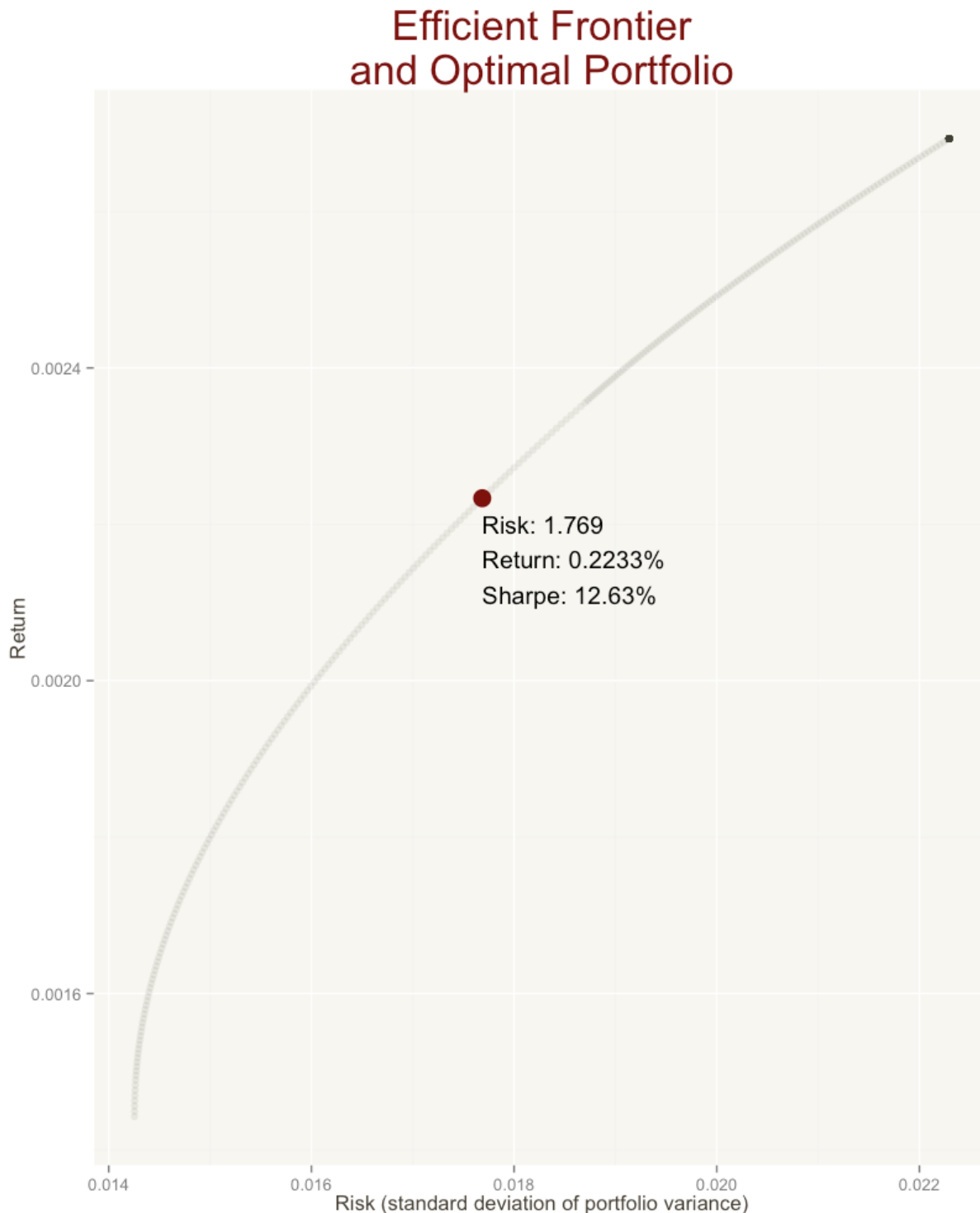
```

```

12 round(eff.optimal.point$Exp.Return*100, digits=4), "%\nSharpe: ",
13 round(eff.optimal.point$sharpe*100, digits=2), "%", sep=""), hjust=
14 ggtitle("Efficient Frontier\nand Optimal Portfolio") + labs(x="Risk
15 theme(panel.background=element_rect(fill=eallighttan), text=element
16 plot.title=element_text(size=24, color=ealred))

```

After copying the preceding code into your R console, you will generate the following chart.



Please watch the following video for a demonstration of using this code to quickly plot the efficient frontier.

R Series - Using R to Plot Efficient Frontier



If you appreciated this article, please take a few seconds to 'like' us below. Thank you!



Like 50

Share 32



Tweet

